1 Roteiro Prático 05 (Parte 1) — Assincronismo, Fetch API & Microbackend Flask

Objetivos de Aprendizagem — Roteiro 05 (Parte 1: Fundamentos & Mini-Exemplos)

Objetivos de Aprendizagem (Módulo 3 — Assincronismo & Integração de Serviços Web)
Ao final desta prática orientada em laboratório, o estudante será capaz de:

- Compreender a transição do armazenamento isolado no navegador (`localStorage`) para a **arquitetura Cliente-Servidor real**, onde os dados são centralizados e compartilhados em rede;
- Compreender o funcionamento interno e o ciclo de vida de requisições assíncronas utilizando a moderna **Fetch API** nativa com `async/await` e Promises;
- Instalar, configurar e executar microbackends em **Python Flask**, entendendo o papel das rotas HTTP, verbos fundamentais (`GET`, `POST`) e serialização de dados em formato JSON;
- Compreender a diferença arquitetural entre frameworks leves de microsserviços (como o Flask) e pilhas tradicionais completas com servidores de aplicação integrados (como o Apache/XAMPP);
- Identificar e resolver o mecanismo de proteção de segurança **CORS** (*Cross-Origin Resource Sharing*), compreendendo por que o navegador bloqueia chamadas entre origens distintas e como liberar o acesso com `flask-cors`;
- Implementar tratamento defensivo de erros de rede utilizando blocos `try/catch/finally`, garantindo resiliência quando o servidor estiver indisponível;
- Desenvolver interfaces reativas capazes de fornecer feedback visual imediato ao usuário através de **Spinners de Carregamento** e desativação temporária de controles.

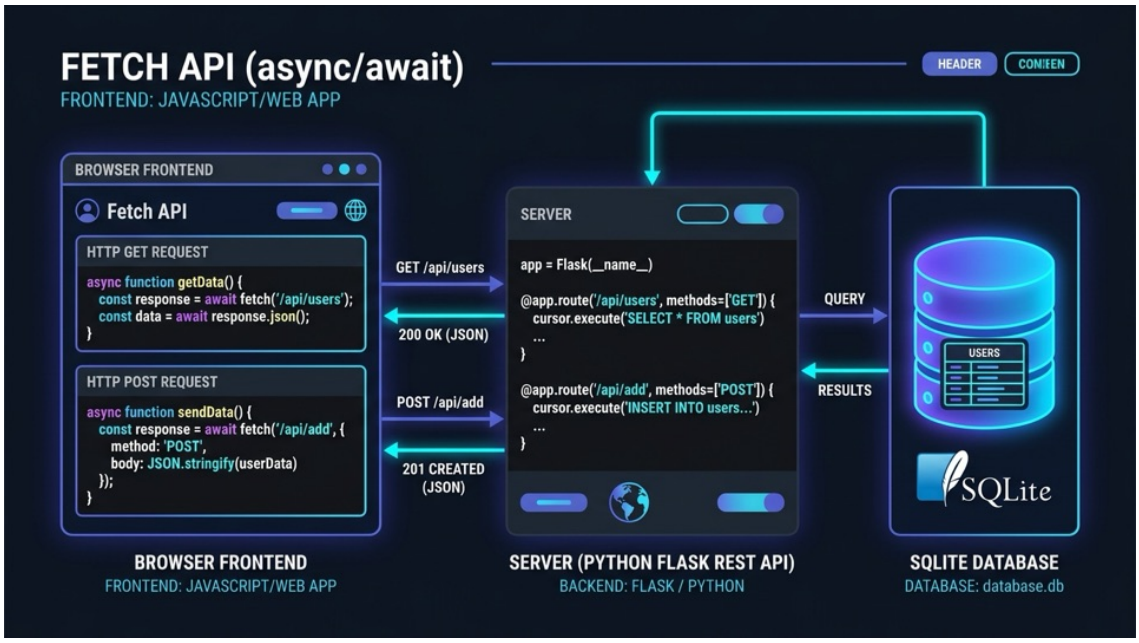


Figura 1: Visão Geral da Aplicação: Vitrine Comunitária conectada via Fetch API com backend RESTful em Flask e persistência relacional SQLite.

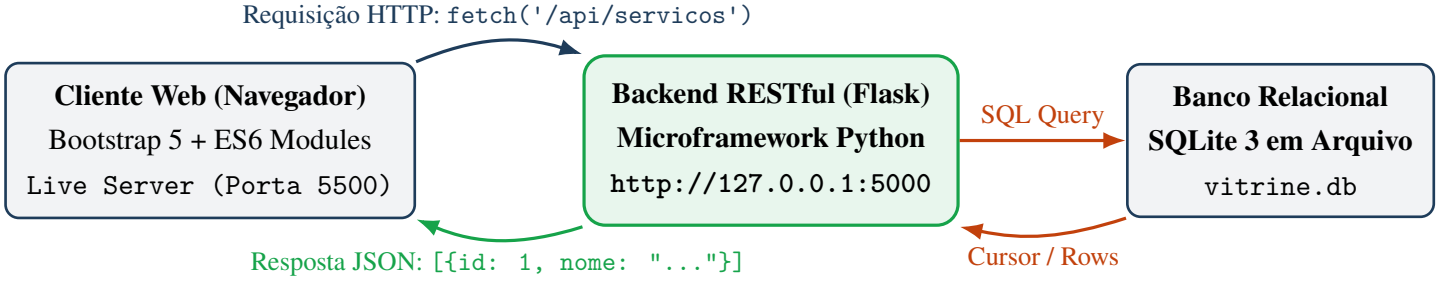
1.1 Introdução: Da Vitrine Isolada à Aplicação em Rede

No **Roteiro Prático 04**, demos um grande salto organizando o frontend na arquitetura modular com ES6 Modules (`services/`, `views/`, `utils/`). No entanto, todo o armazenamento ocorria exclusivamente dentro do navegador de cada máquina através da `Web Storage API` (`localStorage`).

Na prática profissional, se um empreendedor em Garopaba cadastrar um novo serviço em seu celular, esse serviço precisa ficar imediatamente disponível para qualquer cliente que abrir a página em outro computador. Isso exige que os dados saiam do navegador e sejam gravados em um **servidor central com banco de dados**.

Neste **Roteiro Prático 05**, evoluiremos a mesma base de código da **Vitrine Comunitária**, conectando o frontend modular a uma API RESTful leve construída em **Flask** com banco de dados relacional **SQLite**.

1.2 Visão Geral da Arquitetura Cliente-Servidor



1.3 Estrutura de Pastas do Projeto Integrado

Para manter o desacoplamento saudável entre a camada de backend e a camada de frontend, organizaremos o projeto na pasta `roteiro-5/exercicio-roteiro5/` com a seguinte estrutura:

Árvore de Diretórios do Projeto Integrado (Full-Stack Didático):

```
roteiro-5/exercicio-roteiro5/
|-- backend/
|   |-- requirements.txt   <-- Dependências Python (Flask, flask-cors)
|   |-- app.py             <-- API RESTful com rotas e conexão SQLite
|   `-- vitrine.db         <-- Banco SQLite (gerado automaticamente no 1º run)
`-- frontend/
    |-- index.html         <-- Interface com status de conexão e spinner
    |-- styles.css         <-- Estilos complementares
    `-- js/
        |-- main.js        <-- Orquestrador assíncrono de eventos
        |-- services/
        |   `-- vitrineService.js <-- Consumo Fetch API com fallback
        |-- views/
        |   `-- vitrineView.js    <-- Renderizador de cards, spinner e toasts
        `-- utils/
            `-- formatters.js     <-- Formatações (100% inalterado!)
```

Princípio do Desacoplamento Arquitetural

Observe atentamente que a camada utilitária (`formatters.js`) e grande parte da camada de visão (`vitrineView.js`) **permanecem exatamente as mesmas do Roteiro 04**. Toda a migração de tecnologia ocorre de forma isolada no `vitrineService.js`. Este é o maior benefício do padrão arquitetural que aprendemos!

1.4 Conceitos Fundamentais em Pílulas Práticas de Aquecimento

Antes de iniciarmos a construção da arquitetura completa da Vitrine Comunitária (que integrará banco de dados relacional persistente, camadas modulares em ES6 e contingência offline), realizaremos **4 mini-práticas guiadas de aquecimento**.

Cada pílula introduz um conceito arquitetural isolado e fornece o código **100% completo** para ser executado imediatamente pelos estudantes no laboratório em poucos minutos.

Padrão Arquitetural: Separação de Arquivos e Ciclo de Vida do DOM

1. Separação de Responsabilidades (HTML vs. JS): Seguindo a mesma lógica pedagógica aplicada ao longo do curso (Roteiro 02 com manipulação de eventos, Roteiro 04 com camadas modulares), manteremos os arquivos JavaScript estritamente separados em arquivos externos dedicados (`functions.js`). O arquivo `index.html` gerencia apenas a marcação estrutural e folhas de estilo, enquanto `functions.js` concentra a lógica comportamental e requisições assíncronas.

2. O Desafio do Ciclo de Vida do DOM (*Race Conditions*): Quando incluímos um script no cabeçalho (`<head>`) de forma tradicional, o navegador interrompe o processamento do documento HTML para baixar e executar o arquivo JavaScript imediatamente. Nesse momento, os elementos visuais da interface (como `<button id="btnCarregar">` e formulários) **ainda não existem na árvore do DOM**. Caso o script tente acessá-los diretamente, o retorno será `null`, resultando no erro clássico:

```
Uncaught TypeError: Cannot read properties of null (reading
'addEventListener')
```

3. A Solução em Duas Camadas (`defer` + `DOMContentLoaded`): Para garantir um ciclo de vida 100% seguro e resiliente:

- **No HTML:** Utilizamos o atributo **`defer`** na tag de inclusão: `<script src="functions.js" defer></script>`. O `defer` instrui o navegador a carregar o script em segundo plano sem travar o HTML, adiando sua execução até que o parse do documento seja concluído.
- **No JavaScript:** Envolvemos o mapeamento dos nós e a vinculação de ouvintes no evento **`DOMContentLoaded`**:

```
document.addEventListener('DOMContentLoaded', () => {
    // Nós mapeados e eventos registrados com a árvore DOM 100% pronta!
});
```

1.4.1 Pílula 1: Assincronismo e Leitura de Dados com Fetch API (GET)

Fundamentação Teórica: No JavaScript tradicional, operações de rede não podem ser executadas de forma síncrona (*bloqueante*), pois o navegador congelaria a interface enquanto aguardasse a resposta de outro computador.

Por isso, o JavaScript moderno utiliza o mecanismo de **Promises** (promessas de valores futuros) manipuladas pelas palavras-chave **`async`** e **`await`**. A função global **`fetch(url)`** dispara uma requisição HTTP assíncrona. Quando a resposta do servidor chega, utilizamos o método **`await`**

`resposta.json()` para converter o fluxo de texto HTTP em um objeto ou vetor JavaScript utilizável.

Mini-Exemplo 1: Leitura Assíncrona de Arquivo JSON com Fetch GET

Objetivo Prático: Ler um vetor de serviços em formato JSON e preencher dinamicamente uma lista HTML na tela sem recarregar a página, com script externo e controle do DOM.

Passo 1.1: Criar o arquivo de dados (`mini-exemplos/01-fetch-get/dados.json`):

```
[
  { "id": 1, "nome": "Maré Alta Artesanatos", "preco": 35.00 },
  { "id": 2, "nome": "Garopaba Web Studio", "preco": 150.00 },
  { "id": 3, "nome": "Pescado Fresco do Zequinha", "preco": 42.00 }
]
```

Passo 1.2: Criar a página HTML estrutural (`mini-exemplos/01-fetch-get/index.html`):

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mini-Exemplo 1: Fetch API GET</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet"
    >
  <!-- Script externo com defer para garantir o carregamento seguro do DOM -->
  <script src="functions.js" defer></script>
</head>
<body class="bg-light p-4">
  <div class="container" style="max-width: 500px;">
    <div class="card shadow-sm">
      <div class="card-body">
        <h5 class="card-title mb-3">Mini-Exemplo 1: Fetch API (GET)</h5>
        <p class="text-muted small">Carregamento assíncrono de dados com <code>async/await</code>.</p>

        <button id="btnCarregar" class="btn btn-primary w-100 mb-3">
          Carregar Serviços Locais
        </button>

        <ul id="listaServicos" class="list-group">
          <!-- Os itens serão inseridos aqui pelo JavaScript -->
        </ul>
      </div>
    </div>
  </div>
</body>
</html>
```

Passo 1.3: Criar a lógica assíncrona desacoplada (`mini-exemplos/01-fetch-get/functions.js`):

```
// 1. Função assíncrona dedicada com nome claro
async function carregarServicos(listaServicos) {
  try {
    // Passo 1.1: Dispara a requisição HTTP assíncrona para o arquivo
    const resposta = await fetch('dados.json');

    // Passo 1.2: Converte a resposta em formato JSON para um vetor JavaScript
    const servicos = await resposta.json();

    // Passo 1.3: Limpa a lista antes de adicionar novos elementos
    listaServicos.innerHTML = '';

    // Passo 1.4: Percorre a lista e cria os elementos no DOM
    for (const servico of servicos) {
      const item = document.createElement('li');
      item.className = 'list-group-item d-flex justify-content-between align-items-center';
      item.innerHTML = `
        <span>${servico.nome}</span>
        <span class="badge bg-success rounded-pill">R$ ${servico.preco.toFixed(2)}</span>
      `;
      listaServicos.appendChild(item);
    }

  } catch (erro) {
    console.error('Falha ao carregar os dados do arquivo:', erro);
  }
}

// 2. Controle do ciclo de vida: executa quando o DOM estiver completamente pronto
document.addEventListener('DOMContentLoaded', () => {
  const btnCarregar = document.getElementById('btnCarregar');
  const listaServicos = document.getElementById('listaServicos');

  btnCarregar.addEventListener('click', () => carregarServicos(listaServicos));
});
```

Como testar a Pílula 1

Abra o arquivo `index.html` com a extensão **Live Server** do VS Code. Clique no botão **"Carregar Serviços Locais"** e observe os 3 itens serem renderizados instantaneamente no DOM. Abra o Console do DevTools (F12) e confirme que nenhum erro de nó nulo ocorre.

1.4.2 Pílula 2: Microbackend Flask e a Política de Mesma Origem (CORS)

Fundamentação Teórica: O que é o Microframework Flask e Como Funciona

Em uma arquitetura web desacoplada moderna, os dados não residem em arquivos estáticos locais no navegador, mas sim em um servidor backend remoto que expõe uma API RESTful. Na linguagem Python, uma das ferramentas mais consagradas e eficientes para essa finalidade é o microframework **Flask**.

1. O que é o Flask? Criado em 2010 por Armin Ronacher (mantido pelo coletivo de código aberto *Pallets Projects*), o Flask é classificado como um **microframework web**. O termo *"micro"* não significa que ele seja limitado ou frágil; pelo contrário, expressa uma filosofia de design intencional: manter um **núcleo enxuto e minimalista** (*lean core*), sem impor decisões arquiteturais rígidas ao desenvolvedor.

- **Contraste Didático (Flask vs. Django):** Enquanto frameworks full-stack monolíticos (como o Django) seguem a filosofia *"baterias inclusas"* — trazendo nativamente camadas obrigatórias de ORM, painéis de administração prontos e geradores de formulários complexos —, o Flask entrega apenas o estritamente indispensável: o motor de roteamento e despacho HTTP (baseado na biblioteca *Werkzeug*) e o motor de renderização de modelos (*Jinja2*). O programador é 100% livre para escolher seu banco de dados, bibliotecas de validação e arquitetura de pastas.

2. Para quais tipos de aplicações o Flask serve? Pela sua leveza, rapidez de inicialização e baixo consumo de recursos, o Flask é a escolha ideal para:

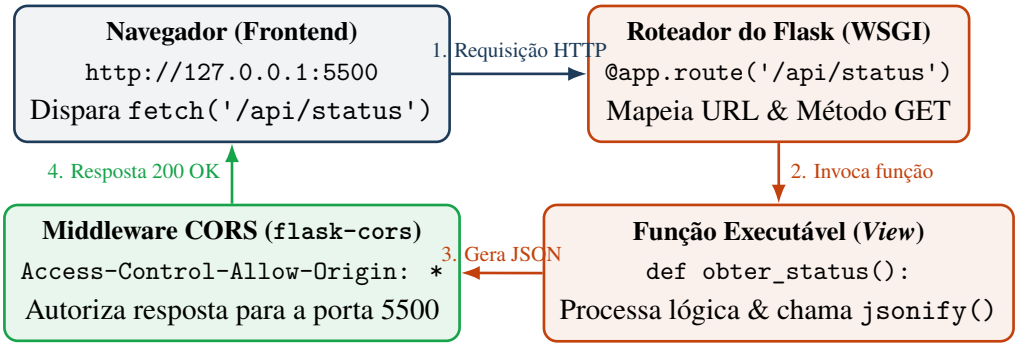
- **APIs RESTful desacopladas:** Prover endpoints JSON consumidos por aplicações web ricas (*Single Page Applications* - SPAs em React, Vue, Angular ou JavaScript Puro) e aplicativos mobile.
- **Microsserviços:** Componentes distribuídos especializados que resolvem tarefas pontuais em nuvem (ex: serviço de envio de e-mails, processamento de pagamentos).
- **Prototipagem Ágil e MVPs (*Minimum Viable Products*):** Permite construir uma API comercial completa em poucos minutos em um único arquivo `.py`.
- **Serviços de Inteligência Artificial e Ciência de Dados:** É o padrão de mercado para empacotar modelos de *Machine Learning* e *Deep Learning* (Scikit-Learn, PyTorch, TensorFlow) e disponibilizá-los na internet como serviços preditivos consumíveis via HTTP.

3. Como o Flask funciona (Arquitetura e Ciclo de Vida)? O Flask baseia-se na especificação padrão **WSGI** (*Web Server Gateway Interface* - PEP 3333), que define como um servidor web se comunica com programas Python:

1. **Instanciação da Aplicação:** Cria-se o objeto central via `app = Flask(__name__)`.
2. **Mapeamento Declarativo com Decoradores:** As rotas da API são declaradas através de decoradores Python: `@app.route('/caminho', methods=['GET', 'POST'])`. Esse padrão associa um endereço URL e um verbo HTTP diretamente a uma função executável (*view function*).

3. **Ciclo de Requisição e Resposta:** Quando o navegador dispara uma requisição HTTP, o Flask a intercepta, processa parâmetros da URL ou corpo (JSON), executa a função mapeada e despacha uma resposta HTTP padronizada de volta ao cliente.

4. **Serialização JSON Nativa (`jsonify`):** Em APIs REST, as respostas são trafegadas em formato JSON. O utilitário nativo `jsonify()` converte dicionários e listas Python em strings formatadas em JSON e anexa automaticamente o cabeçalho `Content-Type: application/json; charset=utf-8`.



4. Comparativo Didático: Pilha Tradicional XAMPP (PHP + Apache + MySQL) vs. Microframework Moderno (Flask + Python + SQLite)

Muitos estudantes de desenvolvimento web iniciam seu aprendizado utilizando o pacote integrado **XAMPP** (*Cross-Platform, Apache, MySQL, PHP, Perl*), que foi o padrão clássico da web durante décadas (e comum em disciplinas introdutórias de banco de dados e programação). Para compreender por que hoje utilizamos o **Flask** com a **Fetch API**, é fundamental contrastar essas duas abordagens:

- **Servidor Web & Inicialização:** No modelo tradicional, o desenvolvedor precisa instalar o software XAMPP no sistema operacional, abrir seu painel de controle e iniciar o servidor **Apache** (que roda em segundo plano consumindo portas do sistema operacional como a 80 e a 443). No Flask, **não é necessário instalar nenhum servidor web externo**: o próprio interpretador Python cria um servidor HTTP de desenvolvimento embutido e de inicialização instantânea através do comando `app.run(port=5000)`.

- **Roteamento Físico vs. Roteamento Declarativo:** No PHP tradicional sobre o Apache, o roteamento de URLs está rigidamente atrelado à **árvore física de arquivos no disco**. Para acessar uma rota, a URL deve espelhar os diretórios do servidor (por exemplo, `http://localhost/meu-app/admin/listar` precisa obrigatoriamente apontar para o arquivo físico `listar.php`). No Flask, o roteamento é **lógico, semântico e desacoplado do sistema de arquivos**: decoradores como `@app.route('/api/produtos')` definem endereços elegantes independentemente de como o código está organizado internamente.

- **Monólito Acoplado vs. Arquitetura Desacoplada (API REST):** No fluxo clássico com PHP/-XAMPP, o backend frequentemente realiza *Server-Side Rendering* (renderização no servidor), mesclando lógica com código de apresentação (`echo '<tr><td>' . $dado . '</td></tr>'`). Toda ação do usuário dispara um recarregamento completo da página web. Na arquitetura moderna que

estamos construindo com Flask, o backend funciona exclusivamente como uma **API REST de dados puros (JSON)**; a interface visual é uma aplicação web rica (SPA) que roda no navegador do usuário e consome os dados de forma assíncrona (`fetch()`) sem recarregar a tela.

- **Banco de Dados & Administração:** No XAMPP, utiliza-se o servidor de banco de dados MySQL/-MariaDB (processo de rede separado) administrado via interface gráfica **phpMyAdmin**. No nosso projeto com Flask, adotamos o **SQLite 3**, cujo banco reside em um único arquivo local (`vitrine.db`) nativo da biblioteca padrão do Python, garantindo portabilidade absoluta (basta mover a pasta do projeto para levar o banco junto, sem senhas, usuários ou portas de rede para configurar).

Aspecto	Pilha Tradicional (XAMPP / PHP)	Microframework Moderno (Flask / Python)
Servidor Web	Apache como serviço externo do SO (painel XAMPP, portas 80/443).	Servidor HTTP embutido no script Python (`app.run(port=5000)`).
Roteamento	Acoplado à árvore física de arquivos no disco (`/script.php`).	Declarativo, semântico e desacoplado (`@app.route('/api/...')`).
Formato de Saída	Renderização no servidor misturando lógica e HTML (*SSR Monolítico*).	API REST desacoplada retornando dados em JSON puro (`jsonify()`).
Banco de Dados	MySQL em rede gerenciado via navegador no phpMyAdmin.	SQLite embutido em arquivo único (`vitrine.db`), zero configuração externa.
Experiência no Cliente	Recarregamento síncrono da página inteira no navegador.	Requisições assíncronas em segundo plano via Fetch API (`async/await`).

Tabela 1: Quadro comparativo entre a abordagem clássica com XAMPP e a arquitetura moderna com Flask.

5. Como verificar se o Flask está instalado no computador?

Antes de iniciar o código prático, verifique se o interpretador Python e o pacote Flask já estão disponíveis no terminal integrado do VS Code (ou Prompt de Comando / Terminal do seu sistema operacional):

```
# No Windows (PowerShell ou Prompt de Comando):
python -m flask --version

# No macOS ou Linux:
python3 -m flask --version
```

Se o Flask já estiver instalado, você verá uma mensagem informando a versão do Python, do Flask (ex: `Flask 3.x.x`) e do Werkzeug.

6. Como instalar o Flask e a biblioteca de CORS (caso não esteja instalado):

Caso o terminal acuse erro de comando não encontrado ou `No module named flask`, execute o seguinte comando para realizar a instalação no seu ambiente:

```
# Instalação do Flask e do middleware CORS via pip:
pip install flask flask-cors

# Ou vinculando explicitamente ao interpretador Python:
python -m pip install flask flask-cors     # Windows
python3 -m pip install flask flask-cors    # macOS / Linux
```

Por que instalamos o `flask-cors`? Por medidas de segurança vitais, os navegadores implementam a **Política de Mesma Origem** (*Same-Origin Policy - SOP*). Como a nossa página frontend roda na porta `5500` (Live Server) e o Flask escuta na porta `5000`, o navegador bloqueia a requisição a menos que o backend declare explicitamente a liberação de **CORS** (*Cross-Origin Resource Sharing*) via cabeçalho `Access-Control-Allow-Origin: *`. A biblioteca `flask-cors` fornece o middleware que habilita essa autorização em apenas uma linha no Python: `CORS(app)`.

Mini-Exemplo 2: Microservidor Flask & Consumo Cross-Origin

Objetivo Prático: Executar um microservidor Python em porta distinta e consumi-lo via Fetch a partir do navegador, compreendendo a atuação do CORS com script desacoplado.

Passo 2.1: Criar o microservidor Python (`mini-exemplos/02-micro-flask-cors/server.py`):

```python
from flask import Flask, jsonify
from flask_cors import CORS

app = Flask(__name__)
# Habilita o CORS para permitir requisições de outras origens (porta 5500)
CORS(app)

@app.route('/api/status', methods=['GET'])
def obter_status():
    return jsonify({
        "status": "online",
        "servidor": "Micro API Flask",
        "cidade": "Garopaba - SC"
    })

if __name__ == '__main__':
    # Executa na porta 5000 com recarga automática
    app.run(port=5000, debug=True)
```

Passo 2.2: Criar a página consumidora (`mini-exemplos/02-micro-flask-cors/index.html`):

```html
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mini-Exemplo 2: Flask API & CORS</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet"
    >
  <!-- Script externo com defer para carregamento seguro -->
  <script src="functions.js" defer></script>
</head>
<body class="bg-light p-4">
  <div class="container" style="max-width: 500px;">
    <div class="card shadow-sm">
      <div class="card-body">
        <h5 class="card-title mb-3">Mini-Exemplo 2: Flask API & CORS</h5>
        <p class="text-muted small">Consumindo um microbackend Python Flask na porta 5000 a partir da
    porta 5500.</p>

        <button id="btnConectar" class="btn btn-dark w-100 mb-3">
          Verificar Status da API
        </button>

        <div id="resultadoStatus" class="alert alert-secondary mb-0">
          Aguardando conexão com a API...
        </div>
      </div>
```

```
    </div>
  </div>
</body>
</html>
```

Passo 2.3: Criar a lógica de consumo assíncrono (`mini-exemplos/02-micro-flask-cors/functions.`

```
// 1. Função assíncrona nomeada que consulta a API do Flask
async function consultarStatusApi(resultadoStatus) {
  try {
    const resposta = await fetch('http://127.0.0.1:5000/api/status');
    const dados = await resposta.json();

    resultadoStatus.className = 'alert alert-success mb-0';
    resultadoStatus.innerHTML = `
      <strong>Status:</strong> ${dados.status} <br>
      <strong>Servidor:</strong> ${dados.servidor} <br>
      <strong>Origem:</strong> ${dados.cidade}
    `;
  } catch (erro) {
    resultadoStatus.className = 'alert alert-danger mb-0';
    resultadoStatus.innerHTML = `
      <strong>Erro de Conexão:</strong> Não foi possível acessar a API.<br>
      <small>Verifique se o server.py está rodando ou se houve bloqueio de CORS.</small>
    `;
  }
}

// 2. Controle seguro de carregamento do DOM
document.addEventListener('DOMContentLoaded', () => {
  const btnConectar = document.getElementById('btnConectar');
  const resultadoStatus = document.getElementById('resultadoStatus');

  btnConectar.addEventListener('click', () => consultarStatusApi(resultadoStatus));
});
```

Como testar a Pílula 2 e observar o CORS em ação

1. No terminal do VS Code, execute: `python server.py`.
2. Abra o `index.html` no Live Server e clique em **"Verificar Status da API"**. O card exibirá a confirmação verde da cidade de Garopaba.
3. **Desafio Didático em Aula:** No `server.py`, comente temporariamente a linha `# CORS(app)` e reinicie o servidor. Ao clicar no botão, abra a aba *Console* do DevTools (F12) e observe o bloqueio em vermelho por violação de CORS! Em seguida, descomente a linha.

1.4.3 Pílula 3: Envio de Dados com Fetch POST e JSON

Fundamentação Teórica: Enquanto requisições **GET** servem para recuperar dados, o método HTTP **POST** transfere novas informações do cliente para persistência no servidor.

Para enviar dados em formato JSON através da Fetch API, três etapas são mandatórias no objeto de configuração:

1. Declarar o método: `method: 'POST'`;
2. Declarar o cabeçalho HTTP: `headers: { 'Content-Type': 'application/json' }`, avisando ao Flask que o corpo trafega em formato JSON;
3. Serializar o objeto JavaScript em texto JSON puro: `body: JSON.stringify(dados)`.

Mini-Exemplo 3: Cadastro de Serviços com Fetch POST

Objetivo Prático: Submeter um formulário HTML e enviar os dados para gravação na API Flask via requisição POST, com validação e script desacoplado.

Passo 3.1: Criar o servidor com rota POST (`mini-exemplos/03-fetch-post/server.py`):

```python
from flask import Flask, jsonify, request
from flask_cors import CORS

app = Flask(__name__)
CORS(app)

servicos_cadastrados = []

@app.route('/api/cadastrar', methods=['POST'])
def cadastrar_servico():
    dados = request.get_json()

    if not dados or 'nome' not in dados or 'preco' not in dados:
        return jsonify({"erro": "Campos obrigatórios ausentes."}), 400

    novo_id = len(servicos_cadastrados) + 1
    registro = {
        "id": novo_id,
        "nome": dados['nome'],
        "preco": float(dados['preco'])
    }
    servicos_cadastrados.append(registro)
    print(f"[API FLASK] Novo serviço cadastrado: {registro}")

    return jsonify({
        "mensagem": "Serviço cadastrado com sucesso no servidor!",
        "registro": registro,
        "total": len(servicos_cadastrados)
    }), 201

if __name__ == '__main__':
    app.run(port=5000, debug=True)
```

Passo 3.2: Criar o formulário frontend (`mini-exemplos/03-fetch-post/index.html`):

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mini-Exemplo 3: Fetch API POST</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet"
    >
  <!-- Script externo com defer para carregamento seguro -->
  <script src="functions.js" defer></script>
</head>
<body class="bg-light p-4">
  <div class="container" style="max-width: 500px;">
    <div class="card shadow-sm">
      <div class="card-body">
        <h5 class="card-title mb-3">Mini-Exemplo 3: Fetch API (POST)</h5>
        <p class="text-muted small">Enviando dados estruturados em JSON via requisição POST.</p>

        <form id="formCadastro">
          <div class="mb-3">
            <label for="campoNome" class="form-label">Nome do Serviço:</label>
            <input type="text" id="campoNome" class="form-control" placeholder="Ex: Conserto de
    Pranchas" required>
          </div>

          <div class="mb-3">
            <label for="campoPreco" class="form-label">Preço Base (R$):</label>
            <input type="number" id="campoPreco" class="form-control" step="0.01" placeholder="Ex:
    80.00" required>
          </div>

          <button type="submit" class="btn btn-success w-100 mb-3">
            Enviar via POST para a API
          </button>
        </form>

        <div id="feedbackPost" class="alert alert-secondary d-none mb-0"></div>
      </div>
    </div>
  </div>
</body>
</html>
```

Passo 3.3: Criar a lógica de envio assíncrono (`mini-exemplos/03-fetch-post/functions.js`):

```
// 1. Função assíncrona nomeada para processar o envio
async function cadastrarNovoServico(evento, formCadastro, campoNome, campoPreco, feedbackPost) {
  evento.preventDefault();

  const dadosParaEnvio = {
    nome: campoNome.value.trim(),
    preco: parseFloat(campoPreco.value)
  };

  try {
```

```
    const resposta = await fetch('http://127.0.0.1:5000/api/cadastrar', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify(dadosParaEnvio)
    });

    // Validação explícita sem contrações
    if (resposta.ok === false) {
      throw new Error(`O servidor retornou o código de erro: ${resposta.status}`);
    }

    const dadosResposta = await resposta.json();

    feedbackPost.className = 'alert alert-success mb-0';
    feedbackPost.classList.remove('d-none');
    feedbackPost.innerHTML = `
      <strong>${dadosResposta.mensagem}</strong><br>
      <small>ID gerado: #${dadosResposta.registro.id} | Total no servidor: ${dadosResposta.total}</
     small>
    `;

    formCadastro.reset();

  } catch (erro) {
    feedbackPost.className = 'alert alert-danger mb-0';
    feedbackPost.classList.remove('d-none');
    feedbackPost.innerHTML = `<strong>Falha ao enviar:</strong> ${erro.message}`;
  }
}

// 2. Controle seguro de carregamento do DOM
document.addEventListener('DOMContentLoaded', () => {
  const formCadastro = document.getElementById('formCadastro');
  const campoNome = document.getElementById('campoNome');
  const campoPreco = document.getElementById('campoPreco');
  const feedbackPost = document.getElementById('feedbackPost');

  formCadastro.addEventListener('submit', (evento) => {
    cadastrarNovoServico(evento, formCadastro, campoNome, campoPreco, feedbackPost);
  });
});
```

Como testar a Pílula 3

Com o `server.py` em execução, digite um serviço e preço no formulário e clique em **"Enviar via POST"**. Observe que o terminal do Python exibirá o log do registro recebido e a página exibirá o alerta verde com o ID gerado pelo servidor!

1.4.4 Pílula 4: Resiliência de Rede e Feedback Visual (try/catch/finally com Spinner)

Fundamentação Teórica: Em aplicações de produção, falhas de conectividade (servidor fora do ar, oscilação de Wi-Fi, erro 500) são inevitáveis. Uma aplicação de qualidade profissional precisa:

- Interceptar erros de rede com blocos `try...catch`;
- Validar o status HTTP com a propriedade booleana `resposta.ok` (que retorna `true` apenas para códigos HTTP 200–299);
- Fornecer feedback visual instantâneo durante a espera (**Spinner de Carregamento** do Bootstrap);
- Utilizar obrigatoriamente o bloco `finally`, que é executado **sempre** (tanto em caso de sucesso quanto de falha), garantindo que o spinner seja ocultado e o botão reativado.

Mini-Exemplo 4: Controle de Loading State e Tratamento de Quedas de Rede

Objetivo Prático: Implementar um botão com spinner de carregamento e tratar quedas de conectividade com feedback amigável ao usuário.

Passo 4.1: Criar a interface resiliente (`mini-exemplos/04-tratamento-erros-spinner/index.html`)

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mini-Exemplo 4: Tratamento de Falhas & Spinners</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet"
    >
  <link href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css" rel="
    stylesheet">
  <!-- Script externo com defer para carregamento seguro -->
  <script src="functions.js" defer></script>
</head>
<body class="bg-light p-4">
  <div class="container" style="max-width: 520px;">
    <div class="card shadow-sm">
      <div class="card-body">
        <h5 class="card-title mb-3">Mini-Exemplo 4: Resiliência &amp; Spinner</h5>
        <p class="text-muted small">
          Controle de estados de carregamento (Loading State) e tratamento resiliente de falhas de rede
     com <code>try...catch...finally</code>.
        </p>

        <button id="btnVerificar" class="btn btn-primary w-100 d-flex justify-content-center align-
    items-center gap-2 mb-3">
          <span id="iconeBotao"><i class="bi bi-arrow-repeat"></i></span>
          <span id="spinnerCarregamento" class="spinner-border spinner-border-sm d-none" role="status">
    </span>
          <span id="textoBotao">Testar Conexão com o Servidor</span>
        </button>

        <div id="caixaAlerta" class="alert d-none mb-0" role="alert"></div>
```

```
      </div>
    </div>
  </div>
</body>
</html>
```

Passo 4.2: Criar as funções de estado e resiliência (`mini-exemplos/04-tratamento-erros-spinner/f`

```
// 1. Funções auxiliares para clareza visual de estados
function ativarEstadoCarregando(elementos) {
  elementos.btnVerificar.disabled = true;
  elementos.iconeBotao.classList.add('d-none');
  elementos.spinnerCarregamento.classList.remove('d-none');
  elementos.textoBotao.textContent = 'Consultando Servidor...';
  elementos.caixaAlerta.classList.add('d-none');
}

function desativarEstadoCarregando(elementos) {
  elementos.spinnerCarregamento.classList.add('d-none');
  elementos.iconeBotao.classList.remove('d-none');
  elementos.textoBotao.textContent = 'Testar Conexão com o Servidor';
  elementos.btnVerificar.disabled = false;
  elementos.caixaAlerta.classList.remove('d-none');
}

// 2. Função assíncrona nomeada com bloco defensivo try...catch...finally
async function testarConexaoComServidor(elementos) {
  ativarEstadoCarregando(elementos);

  try {
    const resposta = await fetch('http://127.0.0.1:5000/api/status');

    if (resposta.ok === false) {
      throw new Error(`O servidor retornou o código de erro HTTP ${resposta.status}.`);
    }

    const dados = await resposta.json();

    elementos.caixaAlerta.className = 'alert alert-success mb-0';
    elementos.caixaAlerta.innerHTML = `
      <i class="bi bi-check-circle-fill me-1"></i>
      <strong>API Online!</strong> Conectado ao servidor (${dados.servidor}).
    `;

  } catch (erro) {
    elementos.caixaAlerta.className = 'alert alert-danger mb-0';
    elementos.caixaAlerta.innerHTML = `
      <i class="bi bi-exclamation-triangle-fill me-1"></i>
      <strong>Falha de Conectividade:</strong> Não foi possível falar com a API.<br>
      <small class="text-muted">Detalhe: ${erro.message}</small>
    `;

  } finally {
    // O bloco finally executa SEMPRE, garantindo restauração do botão
    desativarEstadoCarregando(elementos);
  }
}

// 3. Controle seguro de carregamento do DOM
```

```
document.addEventListener('DOMContentLoaded', () => {
  const elementos = {
    btnVerificar: document.getElementById('btnVerificar'),
    iconeBotao: document.getElementById('iconeBotao'),
    spinnerCarregamento: document.getElementById('spinnerCarregamento'),
    textoBotao: document.getElementById('textoBotao'),
    caixaAlerta: document.getElementById('caixaAlerta')
  };

  elementos.btnVerificar.addEventListener('click', () => {
    testarConexaoComServidor(elementos);
  });
});
```

Como testar a Resiliência da Pílula 4

1. **Cenário de Sucesso:** Com o `server.py` da Pílula 2 rodando na porta 5000, clique no botão. Observe o spinner girar rapidamente e o alerta verde confirmar o status.

2. **Cenário de Queda de Rede (Servidor Desligado):** Pressione `Ctrl+C` no terminal para derrubar o `server.py`. Clique no botão novamente. Observe que o spinner gira e, graças ao bloco `catch`, a tela **não congela**: um alerta vermelho informa a falha de conectividade e o bloco `finally` restaura o botão para novo teste!

Conclusão da Parte 1 e Continuação na Tarefa Guiada (Parte 2)

Parabéns! Você completou com sucesso todos os fundamentos conceituais e os 4 mini-exemplos de aquecimento do Roteiro 05:

- **Pílula 1:** Fetch API consumindo endpoints públicos assíncronos com JavaScript moderno;
- **Pílula 2:** Microbackend em Python Flask com controle explícito de cabeçalhos CORS;
- **Pílula 3:** Submissão de dados via `POST` com corpo JSON (`application/json`);
- **Pílula 4:** Feedback de carregamento com spinners e tratamento defensivo com `try/catch/finally`.

Próximo Passo: Abra o arquivo **Roteiro Prático 05 (Parte 2) — Tarefa Guiada** (`roteiro-5/tarefa-guiada.pdf`), no qual você aplicará todos esses recursos para refatorar o projeto completo da **Vitrine Comercial de Serviços**, integrando persistência em banco relacional SQLite e modo de contingência offline!